

```
# -*- coding: utf-8 -*-
```

```
"""design task .ipynb
```

Automatically generated by Colab.

Original file is located at

https://colab.research.google.com/drive/1mOxCik4g5il_Zmr0DgeHTZWOGUB7stb

```
"""
```

```
# =====
```

```
# CO5 DESIGN TASK
```

```
# AI SYSTEMS USING PYSPARK RDDs AND INTELLIGENT AGENTS
```

```
# =====
```

```
from pyspark import SparkConf, SparkContext, StorageLevel
```

```
# -----
```

```
# Spark Configuration
```

```
# -----
```

```
conf = SparkConf().setAppName("CO5_AI_Design_Task").setMaster("local[*]")
```

```
sc = SparkContext.getOrCreate(conf)
```

```
# =====
```

```
# Q1. HYBRID AI RECOMMENDATION ENGINE
```

```
# =====
```

```
print("\n===== Q1: HYBRID AI RECOMMENDATION ENGINE =====")
```

```
# Sample user-item-rating data
```

```
ratings = [
```

```

("U1", "I1", 5.0),
("U1", "I2", 4.0),
("U1", "I3", 2.0),
("U2", "I1", 4.0),
("U2", "I2", 5.0),
("U2", "I4", 4.0),
("U3", "I2", 4.0),
("U3", "I3", 5.0),
("U3", "I4", 5.0)
]

ratings_rdd = sc.parallelize(ratings)

# Map user-item interactions
user_item_rdd = ratings_rdd.map(
    lambda x: (x[0], x[1], x[2])
)

# Filter valid ratings
clean_rdd = user_item_rdd.filter(
    lambda x: x[2] > 0
)

# Calculate average rating for each item
item_ratings = ratings_rdd.map(
    lambda x: (x[1], x[2])
)

item_average = item_ratings.groupByKey().mapValues(
    lambda values: sum(values) / len(values)
)

```

```
# Content-based recommendation
```

```
content_features = {  
    "I1": {"action", "adventure"},  
    "I2": {"action", "fantasy"},  
    "I3": {"fantasy", "animation"},  
    "I4": {"action", "fantasy"}  
}
```

```
def content_score(item):  
    preferred = {"action", "fantasy"}  
    features = content_features.get(item, set())  
    return len(preferred.intersection(features))
```

```
content_scores = sc.parallelize(  
    list(content_features.keys())  
)map(  
    lambda item: (item, content_score(item))  
)
```

```
# Collaborative filtering style score
```

```
collaborative_scores = item_average.mapValues(  
    lambda x: x / 5.0  
)
```

```
# Hybrid score
```

```
hybrid_scores = content_scores.join(  
    collaborative_scores  
)mapValues(  
    lambda x: 0.5 * x[0] + 0.5 * x[1]  
)
```

```
recommendations = hybrid_scores.sortBy(  
    lambda x: x[1],  
    ascending=False  
)
```

```
print("Hybrid Recommendations:")  
for item, score in recommendations.collect():  
    print(item, "Score:", round(score, 3))
```

Intelligent Agents

```
class ProfileAgent:  
    def update_profile(self, user, behavior):  
        print("Profile Agent:", user, "behavior updated:", behavior)
```

```
class RecommendationAgent:  
    def generate(self, recommendations):  
        return recommendations
```

```
class FeedbackAgent:  
    def process_feedback(self, user, feedback):  
        print("Feedback Agent:", user, "feedback =", feedback)
```

```
profile_agent = ProfileAgent()  
recommendation_agent = RecommendationAgent()  
feedback_agent = FeedbackAgent()
```

```
profile_agent.update_profile("U1", "Liked I2")
feedback_agent.process_feedback("U1", "Positive")
print("Top Recommendation:",
      recommendation_agent.generate(recommendations.collect())[:2])
```

```
# =====
```

```
# Q2. SMART DISASTER DETECTION SYSTEM
```

```
# =====
```

```
print("\n===== Q2: SMART DISASTER DETECTION =====")
```

```
# Sample multi-source data
```

```
sensor_data = [
    ("Chennai", "water", 90),
    ("Chennai", "rain", 85),
    ("Chennai", "water", 95),
    ("CityA", "water", 30),
    ("CityB", "water", 40)
]
```

```
social_data = [
    ("Chennai", "flood"),
    ("Chennai", "flood"),
    ("CityA", "normal"),
    ("CityB", "normal")
]
```

```
satellite_data = [
    ("Chennai", "flooded_area", 1),
```

```
    ("CityA", "flooded_area", 0),  
    ("CityB", "flooded_area", 0)  
]
```

```
sensor_rdd = sc.parallelize(sensor_data)  
social_rdd = sc.parallelize(social_data)  
satellite_rdd = sc.parallelize(satellite_data)
```

```
# Sensor processing
```

```
clean_sensor = sensor_rdd.filter(  
    lambda x: x[2] >= 0  
)
```

```
water_rdd = clean_sensor.filter(  
    lambda x: x[1] == "water"  
)  
.map(  
    lambda x: (x[0], x[2])  
)
```

```
water_average = water_rdd.groupByKey().mapValues(  
    lambda values: sum(values) / len(values)  
)
```

```
# Social media processing
```

```
social_count = social_rdd.filter(  
    lambda x: x[1] == "flood"  
)  
.map(  
    lambda x: (x[0], 1)  
)  
.reduceByKey(  
    lambda a, b: a + b  
)
```

```
# Satellite processing
```

```
satellite_rdd = satellite_rdd.map(  
    lambda x: (x[0], x[2])  
)
```

```
# Data fusion
```

```
fusion = water_average.join(  
    satellite_rdd  
)  
.join(  
    social_count  
)
```

```
# Disaster detection
```

```
def detect_disaster(data):
```

```
    water_value = data[0][0]
```

```
    satellite_flag = data[0][1]
```

```
    social_reports = data[1]
```

```
    if water_value > 80 and satellite_flag == 1 and social_reports >= 2:
```

```
        return "HIGH FLOOD RISK"
```

```
    elif water_value > 70:
```

```
        return "MEDIUM FLOOD RISK"
```

```
    else:
```

```
        return "LOW RISK"
```

```
disaster_result = fusion.mapValues(detect_disaster)
```

```
print("Disaster Detection Results:")
```

```
for location, result in disaster_result.collect():
```

```
    print(location, ":", result)
```

```
# Intelligent Agents
```

```
class DetectionAgent:
```

```
    def detect(self, result):
```

```
        return result
```

```
class FusionAgent:
```

```
    def confirm(self, result):
```

```
        return result == "HIGH FLOOD RISK"
```

```
class ResponseAgent:
```

```
    def response(self, confirmed):
```

```
        if confirmed:
```

```
            return "SEND EARLY WARNING"
```

```
        return "CONTINUE MONITORING"
```

```
detection_agent = DetectionAgent()
```

```
fusion_agent = FusionAgent()
```

```
response_agent = ResponseAgent()
```

```
for location, result in disaster_result.collect():
```

```
    detected = detection_agent.detect(result)
```

```
    confirmed = fusion_agent.confirm(detected)
```

```
    action = response_agent.response(confirmed)
```

```
    print(location, "->", action)
```



```

# =====

# Q3. AI-BASED FRAUD DETECTION FRAMEWORK

# =====

print("\n===== Q3: AI-BASED FRAUD DETECTION =====")

transactions = [
    ("T1", "U1", 500, "Chennai"),
    ("T2", "U1", 600, "Chennai"),
    ("T3", "U2", 2000, "Bangalore"),
    ("T4", "U3", 250000, "Foreign"),
    ("T5", "U2", 1500, "Bangalore"),
    ("T6", "U4", 300000, "Foreign")
]

transaction_rdd = sc.parallelize(transactions)

# Remove invalid transactions
clean_transactions = transaction_rdd.filter(
    lambda x: x[2] > 0
)

# Extract transaction features
features_rdd = clean_transactions.map(
    lambda x: (
        x[0],
        {
            "user": x[1],
            "amount": x[2],

```

```

        "location": x[3]
    }
)
)

# Simple anomaly detection
def calculate_risk(record):
    transaction_id, data = record

    risk = 0

    if data["amount"] > 100000:
        risk += 70

    if data["location"] == "Foreign":
        risk += 20

    if data["amount"] > 50000:
        risk += 10

    return transaction_id, risk

risk_rdd = features_rdd.map(calculate_risk)

# Classify transactions
fraud_rdd = risk_rdd.map(
    lambda x: (
        x[0],
        x[1],
        "FRAUD" if x[1] >= 70 else "SAFE"
    )
)

```

```
)
```

```
print("Fraud Detection Results:")  
for transaction in fraud_rdd.collect():  
    print(transaction)
```

```
# Intelligent Agents
```

```
class AnomalyAgent:  
    def analyze(self, risk):  
        return risk >= 70
```

```
class AlertAgent:  
    def alert(self, suspicious):  
        if suspicious:  
            return "FRAUD ALERT"  
        return "NO ALERT"
```

```
class LearningAgent:  
    def learn(self, feedback):  
        print("Learning Agent: Model updated using", feedback)
```

```
anomaly_agent = AnomalyAgent()  
alert_agent = AlertAgent()  
learning_agent = LearningAgent()
```

```
for transaction_id, risk, status in fraud_rdd.collect():
```

```
suspicious = anomaly_agent.analyze(risk)
```

```
alert = alert_agent.alert(suspicious)
```

```
print(transaction_id, "->", alert)
```

```
learning_agent.learn("Confirmed fraud cases")
```

```
# =====
```

```
# Q4. AUTONOMOUS SUPPLY CHAIN OPTIMIZATION
```

```
# =====
```

```
print("\n===== Q4: SUPPLY CHAIN OPTIMIZATION =====")
```

```
# Product sales data
```

```
sales = [
```

```
    ("P1", 100),
```

```
    ("P1", 150),
```

```
    ("P1", 200),
```

```
    ("P2", 80),
```

```
    ("P2", 120),
```

```
    ("P3", 200)
```

```
]
```

```
sales_rdd = sc.parallelize(sales)
```

```
# Aggregate sales by product
```

```
total_sales = sales_rdd.reduceByKey(
```

```
    lambda a, b: a + b
```

```
)
```

```
print("Total Sales:")  
for product, value in total_sales.collect():  
    print(product, ":", value)
```

```
# Inventory data
```

```
inventory = [  
    ("P1", 100),  
    ("P2", 150),  
    ("P3", 300)  
]
```

```
inventory_rdd = sc.parallelize(inventory)
```

```
# Join sales and inventory
```

```
supply_data = total_sales.join(  
    inventory_rdd  
)
```

```
# Inventory decision
```

```
def inventory_decision(data):
```

```
    demand = data[0]
```

```
    stock = data[1]
```

```
    if stock < demand:
```

```
        return "REORDER"
```

```
    elif stock > demand * 2:
```

```
        return "REDUCE STOCK"
```

```
    else:
```

```
        return "OPTIMAL"
```

```
inventory_result = supply_data.mapValues(  
    inventory_decision  
)  
  
print("Inventory Decisions:")  
for product, decision in inventory_result.collect():  
    print(product, ":", decision)
```

Intelligent Agents

```
class DemandAgent:  
    def forecast(self, sales):  
        return sales * 1.10
```

```
class InventoryAgent:  
    def decide(self, demand, stock):  
        if stock < demand:  
            return "REORDER"  
        return "STOCK OK"
```

```
class LogisticsAgent:  
    def optimize_route(self, distance):  
        if distance < 100:  
            return "DIRECT ROUTE"  
        return "OPTIMIZED ROUTE"
```

```
class ProcurementAgent:
```

```
def select_supplier(self, price):  
    return "SELECT LOW-COST SUPPLIER"
```

```
demand_agent = DemandAgent()  
inventory_agent = InventoryAgent()  
logistics_agent = LogisticsAgent()  
procurement_agent = ProcurementAgent()  
  
for product, (sales_value, stock) in supply_data.collect():  
  
    forecast = demand_agent.forecast(sales_value)  
    inventory_decision_result = inventory_agent.decide(  
        forecast, stock  
    )  
  
    print(  
        product,  
        "Forecast:", round(forecast, 2),  
        "Decision:", inventory_decision_result  
    )  
  
    print(  
        "Logistics:",  
        logistics_agent.optimize_route(80)  
    )  
  
    print(  
        "Procurement:",  
        procurement_agent.select_supplier(50)  
    )
```

```

# =====

# Q5. PERSONALIZED LEARNING ANALYTICS PLATFORM

# =====

print("\n===== Q5: PERSONALIZED LEARNING ANALYTICS =====")

# Student performance data
students = [
    ("S1", "Python", 85),
    ("S1", "Database", 90),
    ("S1", "AI", 60),
    ("S2", "Python", 55),
    ("S2", "Database", 80),
    ("S2", "AI", 70),
    ("S3", "Python", 95),
    ("S3", "Database", 88),
    ("S3", "AI", 92)
]

student_rdd = sc.parallelize(students)

# Remove invalid scores
clean_students = student_rdd.filter(
    lambda x: 0 <= x[2] <= 100
)

# Group subjects by student
student_subjects = clean_students.groupBy(
    lambda x: x[0]

```



```
)
```

```
# Calculate average score
```

```
student_average = clean_students.map(  
    lambda x: (x[0], x[2])  
)groupByKey().mapValues(  
    lambda scores: sum(scores) / len(scores)  
)
```

```
print("Student Average Scores:")
```

```
for student, score in student_average.collect():  
    print(student, ":", round(score, 2))
```

```
# Identify weak subjects
```

```
weak_subjects = clean_students.filter(  
    lambda x: x[2] < 70  
)
```

```
print("\nWeak Subjects:")
```

```
for student, subject, score in weak_subjects.collect():  
    print(  
        student,  
        "needs improvement in",  
        subject,  
        "Score:",  
        score  
    )
```

Intelligent Agents

class StudentAgent:

def analyze(self, student, score):

if score < 70:

return "Needs Improvement"

elif score < 85:

return "Average"

return "Good"

class ContentAgent:

def recommend(self, status):

if status == "Needs Improvement":

return "Basic lessons + Practice exercises"

elif status == "Average":

return "Intermediate lessons + Quiz"

return "Advanced lessons + Projects"

class FeedbackAgent:

def feedback(self, score):

if score < 70:

return "Practice more and review weak topics"

elif score < 85:

return "Good progress, continue practicing"

return "Excellent performance"

student_agent = StudentAgent()

```

content_agent = ContentAgent()
feedback_agent = FeedbackAgent()

print("\nPersonalized Learning Recommendations:")

for student, score in student_average.collect():

    status = student_agent.analyze(student, score)

    content = content_agent.recommend(status)

    feedback = feedback_agent.feedback(score)

    print("\nStudent:", student)
    print("Performance:", round(score, 2))
    print("Status:", status)
    print("Recommended Content:", content)
    print("Feedback:", feedback)

# =====
# FINAL SUMMARY
# =====

print("\n=====")
print("          CO5 DESIGN TASK COMPLETED")
print("=====")

print("""
Q1 - Hybrid Recommendation Engine
      Content-Based + Collaborative Filtering + Agents

```

Q2 - Smart Disaster Detection

Multi-source Data Fusion + Disaster Agents

Q3 - Fraud Detection

Distributed Anomaly Detection + Fraud Agents

Q4 - Supply Chain Optimization

Demand + Inventory + Logistics + Procurement Agents

Q5 - Personalized Learning

Student Analytics + Adaptive Content + Feedback Agents

PySpark RDD Operations Used:

map()

filter()

flatMap()

groupByKey()

reduceByKey()

join()

persist()

The systems demonstrate:

- Distributed Processing
- Intelligent Agents
- Scalability
- Fault Tolerance
- Real-Time/Adaptive Decision Making

""")

Stop Spark

sc.stop()